

```
In [3]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [4]: import sqlite3
```

```
In [5]: conn = sqlite3.connect('customer_churn.db')

# sql query to Get all table names
sql_query = """
    SELECT name
    FROM sqlite_master
    WHERE type='table';
"""

# read sql query in pandas
tables = pd.read_sql(sql_query, conn)

# create dataframe for each table
for table_name in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table_name}", conn) # Read table into dataframe
    globals()[f"df_{table_name}"] = df # Create dynamic dataframe
    print(f"Created dataframe: df_{table_name}")

# Close connection
conn.close()
```

Created dataframe: df_db_customer

Created dataframe: df_db_subscription

Created dataframe: df_db_support

```
In [6]: import sqlite3
import pandas as pd

conn = sqlite3.connect(r"C:\Users\admin\customer_churn.db")
```

```
In [7]: df_customer = pd.read_sql(
    "SELECT * FROM db_customer",
    conn
)
```

```
In [8]: df_customer.head()
```

```
Out[8]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>name</th> <th>country</th> <th>state</th>
<th>gender</th> <th>dob</th> <th>interests</th> <th>pincode</th> </thead>
<tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody>
<tbody></tbody>
```

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	None
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	NaN	None
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	None
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	NaN	None
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	None

```
In [9]: df_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   customerid  21 non-null    str
1   name        21 non-null    str
2   country     18 non-null    str
3   state       21 non-null    str
4   gender      21 non-null    str
5   dob         21 non-null    str
6   interests   4 non-null     str
7   pincode     0 non-null     object
dtypes: object(1), str(7)
memory usage: 1.4+ KB
```

```
In [10]: df_customer.rename(columns = {'name' : 'customer_name'}, inplace=True)
df_customer
```

```
Out[10]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>customer_name</th> <th>country</th>
<th>state</th> <th>gender</th> <th>dob</th> <th>interests</th>
<th>pincode</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
<th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th>
<th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th>
<th>15</th> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th>
</tbody> <tbody> </tbody>
```

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	None
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	NaN	None
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	None
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	NaN	None
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	None
0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10 00:00:00	NaN	None
0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00	NaN	None
0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00	NaN	None
0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07 00:00:00	NaN	None
0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00	NaN	None
0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00	NaN	None
0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00	NaN	None
0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01 00:00:00	NaN	None
0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00	NaN	None
0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00	NaN	None
0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00	job	None
0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	None

0021- IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	None
0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	None
0023- HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	None
0023- UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	None

```
In [11]: df_customer.drop(columns=['interests','pincode'],inplace=True)
```

```
In [12]: df_customer
```

```
Out[12]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>customer_name</th> <th>country</th>
<th>state</th> <th>gender</th> <th>dob</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th>
<th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th>
<th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th>
<th>19</th> <th>20</th> </tbody> <tbody></tbody>
```

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00
0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10 00:00:00
0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00
0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00
0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07 00:00:00
0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00
0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00
0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00
0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01 00:00:00
0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00
0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00
0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00
0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00
0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00
0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00
0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00
0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00

```
In [13]: df_customer['dob']=pd.to_datetime(df_customer['dob'])
```

```
In [14]: df_customer
```

```
Out[14]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>customer_name</th> <th>country</th>
<th>state</th> <th>gender</th> <th>dob</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th>
<th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th>
<th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th>
<th>19</th> <th>20</th> </tbody> <tbody></tbody>
```

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05
0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10
0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21
0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14
0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07
0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29
0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22
0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18
0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01
0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25
0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11
0020-INWCK	parvati	India	Delhi	Female	1986-02-28
0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19
0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02
0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30
0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14
0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06

```
In [15]: #standardizing gender column
df_customer['gender'].replace({'Men':'Male','Women':'Female'})
```

```
Out[15]: 0      Male
         1      Male
         2    Female
         3      Male
         4    Female
         5    Female
         6    Female
         7      Male
         8    Female
         9      Male
        10      Male
        11    Female
        12    Female
        13    Female
        14      Male
        15    Female
        16    Female
        17    Female
        18      Male
        19      Male
        20    Female
        Name: gender, dtype: str
```

```
In [16]: #fixing missing values
         df_customer[df_customer['country'].isna()]
```

```
Out[16]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>customer_name</th> <th>country</th>
<th>state</th> <th>gender</th> <th>dob</th> </thead> <tbody> <th>5</th>
<th>8</th> <th>12</th> </tbody> <tbody> </tbody>
```

```
0013-MHZWF  durga  NaN      Delhi  Women  1988-12-10
```

```
0015-UOCOJ  maya  NaN  Kathmandu  Women  1985-07-07
```

```
0018-NYROU  chitra  NaN  Telangana  Female  2004-12-01
```

```
In [17]: state_country=df_customer.dropna(subset=['country']).set_index('state')['country']
         df_customer['country']=df_customer['country'].fillna(df_customer['state'].map(stat
```

```
In [18]: df_customer[df_customer['country'].isna()]
```

```
Out[18]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>customer_name</th> <th>country</th>
<th>state</th> <th>gender</th> <th>dob</th> </thead> <tbody> </tbody>
<tbody> </tbody>
```

```
In [19]: df_db_subscription.head()
```

```
Out[19]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>subscription_start_date</th>
<th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th>
<th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th>
<th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> </thead> <tbody>
<th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody>
</tbody>
```

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaN	NaN	13.99	627	12
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaN	NaN	6.99	210	34
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaN	NaN	22.99	1725	8
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88

```
In [20]: date_col = ['subscription_start_date','renewal_date','cancellation_date']
df_db_subscription[date_col]=df_db_subscription[date_col].apply(pd.to_datetime)
```

```
In [21]: df_db_support.head()
```

```
Out[21]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th>
<th>csat_score</th> <th>col_1</th> <th>comment</th> </thead> <tbody>
<th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody>
</tbody>
```

0003-MKNFE	2024-08-28 00:00:00	N	60	None	service issue
0003-MKNFE	2024-08-28 00:00:00	Y	10	None	demaned refund
0013-EXCHZ	2024-01-20 00:00:00	Y	20	None	NaN
0013-MHZWF	2025-03-18 00:00:00	N	90	None	guidance to renew
0013-SMEOE	2024-11-01 00:00:00	N	30	None	NaN

```
In [22]: df_db_support.drop(columns=['col_1','comment'],inplace=True)
```

```
In [23]: df_db_support['complaint_date']=pd.to_datetime(df_db_support['complaint_date'])
```

```
In [24]: #feature engineering
```

```
In [25]: df_db_subscription['Red_flag'] = np.where(df_db_subscription['cancellation_date'],
```

In [26]: `df_db_subscription.head(3)`

Out[26]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>subscription_start_date</th>
<th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th>
<th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th>
<th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> <th>Red_flag</th>
</thead> <tbody> <th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	12
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	NaN	6.99	210	34



In [27]: `df = (df_db_subscription
 .merge(df_customer , on = 'customerid', how='left')
 .merge(df_db_support , on = 'customerid', how='left'))`

In [28]: `df.head()`

Out[28]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>subscription_start_date</th>
<th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th>
<th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th>
<th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> <th>Red_flag</th>
<th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th>
<th>dob</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th>
</thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th>
</tbody> <tbody></tbody>

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	12
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	NaN	6.99	210	34
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	NaN	22.99	1725	8



```
In [29]: df_db_support['complaint_count'] = df_db_support.groupby('customerid')['customerid']
```

```
In [30]: df_db_support.sort_values('complaint_date').drop_duplicates('customerid',keep='last')
```

```
Out[30]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; }.dataframe
tbody tr th { vertical-align: top; }.dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th>
<th>csat_score</th> <th>complaint_count</th> </thead> <tbody> <th>2</th>
<th>5</th> <th>1</th> <th>8</th> <th>6</th> <th>4</th> <th>3</th>
</tbody> <tbody></tbody>
```

```
0013-EXCHZ 2024-01-20 Y 20 1
```

```
0017-IUDMW 2024-04-10 Y 25 1
```

```
0003-MKNFE 2024-08-28 Y 10 2
```

```
0022-TCJCI 2024-09-14 N 90 2
```

```
0019-EFAEP 2024-09-27 Y 30 1
```

```
0013-SMEOE 2024-11-01 N 30 1
```

```
0013-MHZWF 2025-03-18 N 90 1
```

```
In [31]: df = (df_db_subscription
              .merge(df_customer , on = 'customerid', how='left')
              .merge(df_db_support , on = 'customerid', how='left') )
```

```
In [32]: df.shape
```

```
Out[32]: (23, 21)
```

```
In [ ]:
```

```
In [33]: ##data analysis
```

```
In [34]: df.to_csv('export_churn_data.csv', index=False)
```

```
In [35]: churn_rate = df['Red_flag'].mean()*100
print("Churn Rate = " , round(churn_rate,2), "%")
```

Churn Rate = 34.78 %

```
In [36]: retention_rate = 100-churn_rate
print("Retention_rate = " , round(retention_rate,2), "%")
```

Retention_rate = 65.22 %

```
In [37]: plan = df.groupby('plan_type')['Red_flag'].mean().mul(100).round(2).reset_index()
print("churn by plan = " , plan)
```

```
churn by plan =   plan_type  Red_flag
0      Basic    66.67
1    Premium    25.00
2   Standard    22.22
```

```
In [38]: arpu = df['monthly_charges'].mean()
print('ARPU = ', round(arpu,2))
```

ARPU = 18.51

```
In [39]: today = pd.Timestamp.today()

df['tenure_days']=np.where(
    df['cancellation_date'].notna(),

    (df['cancellation_date'] - df['subscription_start_date']).dt.days,
    (today - df['subscription_start_date']).dt.days
)
avg_tenure = df['tenure_days'].mean()
print("Avg tenure (Days) = ", avg_tenure)
```

Avg tenure (Days) = 1495.2173913043478

```
In [40]: revenue_loss = df.loc[df['Red_flag']==1 , 'monthly_charges'].sum()
print("revenue at risk(Rs lakhs) =",revenue_loss)
```

revenue at risk(Rs lakhs) = 103.91999999999999

```
In [41]: escalation_rate = (df['escalations'] == 'Y').mean()*100
print("Escalation Rate = ", round(escalation_rate, 2), "%")
```

Escalation Rate = 21.74 %

```
In [42]: avg_complaint = df['complaint_count'].sum()/df['customerid'].nunique()
print("Avg Complaint = ", round(avg_complaint, 2), "%")
```

Avg Complaint = 0.62 %

```
In [43]: df['escalations'] = np.where(df['escalations'] == 'Y' , 1,0)
corr_df = df[['escalations', 'Red_flag']].dropna()

correlation = corr_df['escalations'].corr(df['Red_flag'])
print("Correlation between escalation vs churn is =" ,round(correlation,2))
```

Correlation between escalation vs churn is = 0.72

```
In [44]: conditions =[
    (df['churn_score']<50),
    (df['churn_score']>=50) & (df['churn_score']<70),
    (df['churn_score']>=70)

]

choices = ['low','med','high']

df['churn_rishk'] = np.select(conditions,choices,default='unknown')
```

```
In [45]: #visualization using matplotlib
```

```
In [46]: df.head()
```

```
Out[46]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>subscription_start_date</th>
<th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th>
<th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th>
<th>monthly_charges</th> <th>cltv</th> <th>...</th> <th>country</th>
<th>state</th> <th>gender</th> <th>dob</th> <th>complaint_date</th>
<th>escalations</th> <th>csat_score</th> <th>complaint_count</th>
<th>tenure_days</th> <th>churn_rishk</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>
```

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	...	Ir
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	...	Ir
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	...	Ir
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	NaN	6.99	210	...	Ir
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	NaN	22.99	1725	...	Ir

<p>5 rows × 23 columns</p>



```
In [47]: df_visual = df.copy()
```

```
In [48]: df_visual.head()
```

```
Out[48]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>subscription_start_date</th>
<th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th>
<th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th>
<th>monthly_charges</th> <th>cltv</th> <th>...</th> <th>country</th>
<th>state</th> <th>gender</th> <th>dob</th> <th>complaint_date</th>
<th>escalations</th> <th>csat_score</th> <th>complaint_count</th>
<th>tenure_days</th> <th>churn_rishk</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>
```

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	...	Ir
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	...	Ir
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	...	Ir
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	NaN	6.99	210	...	Ir
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	NaN	22.99	1725	...	Ir

<p>5 rows × 23 columns</p>



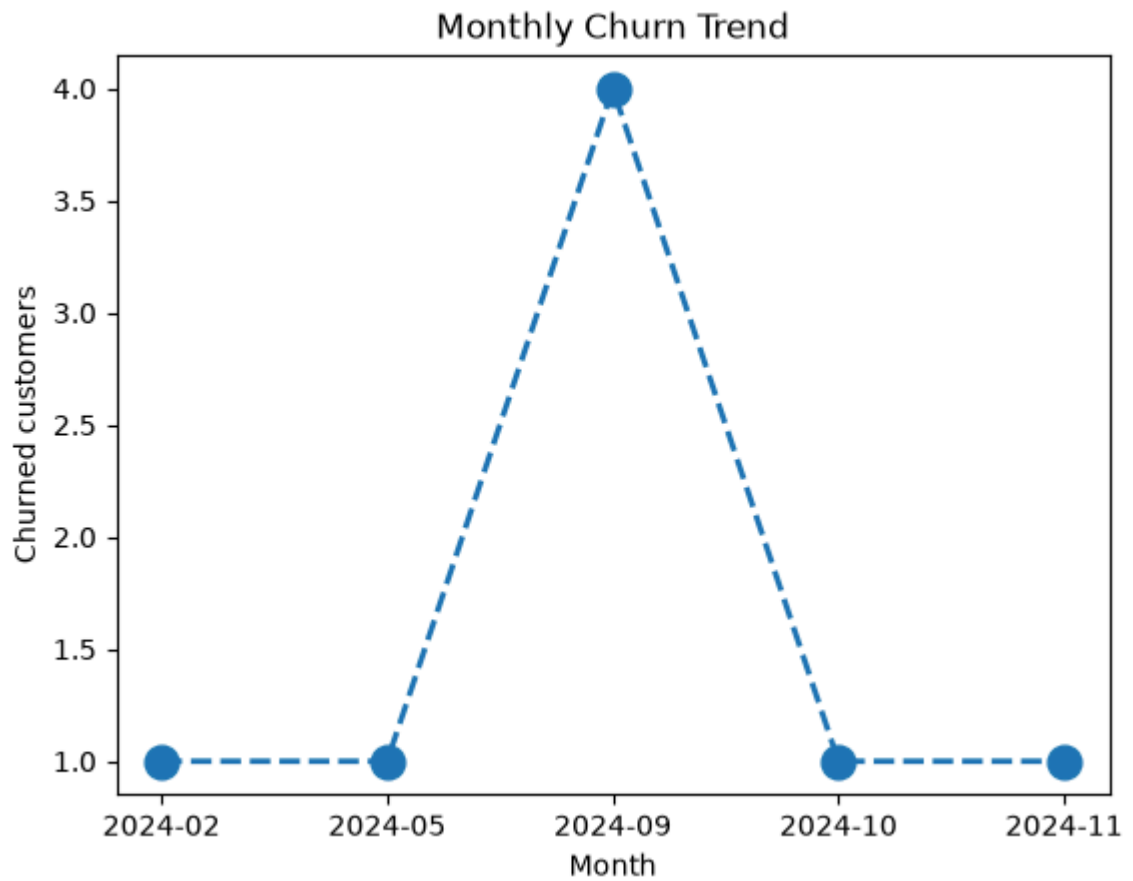
```
In [49]: #monthly churn trend
```

```
In [50]: df_visual['cancellation_month'] = df_visual['cancellation_date'] = df_visual['car
```

```
In [51]: churn_trend = df_visual[df_visual['Red_flag']==1].groupby('cancellation_month').si

plt.plot(churn_trend.index.astype('str'),churn_trend.values , marker='o',linestyle

plt.title('Monthly Churn Trend')
plt.xlabel('Month')
plt.ylabel('Churned customers')
plt.show()
```



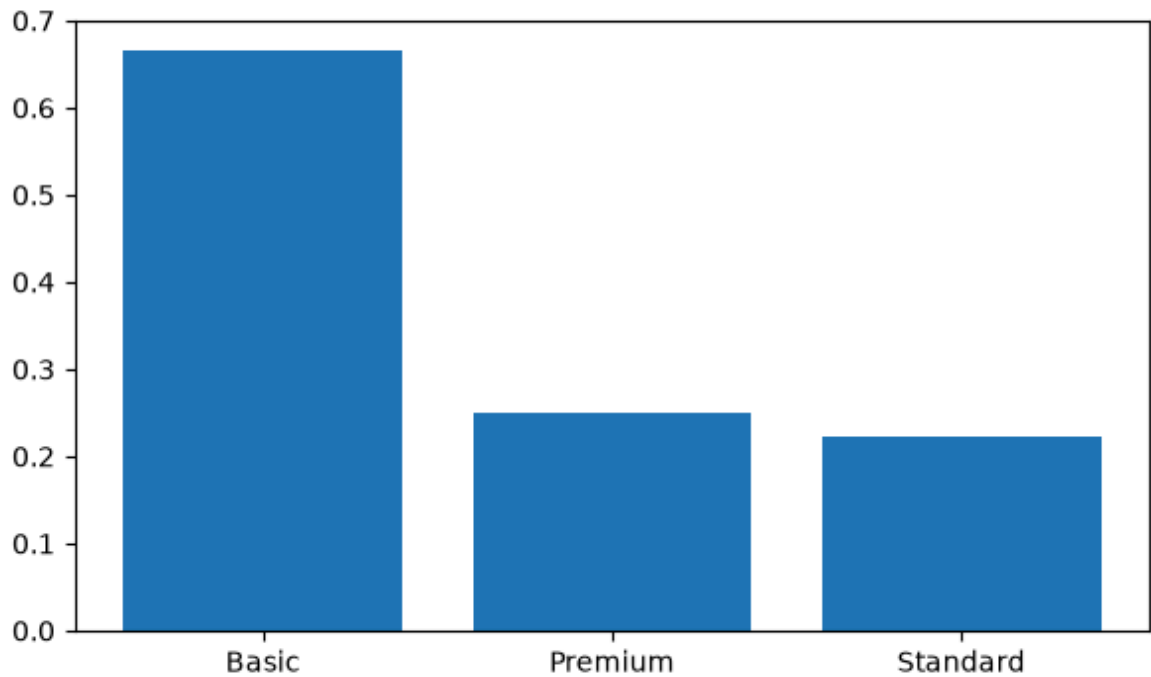
In [52]: `#churn by plan type`

```
In [53]: churn_plan = df_visual.groupby('plan_type')['Red_flag'].mean()
colors = ['red', 'yellow', 'green']

plt.figure(figsize=(7,4))

plt.bar(churn_plan.index , churn_plan.values)
```

Out[53]: <BarContainer object of 3 artists>



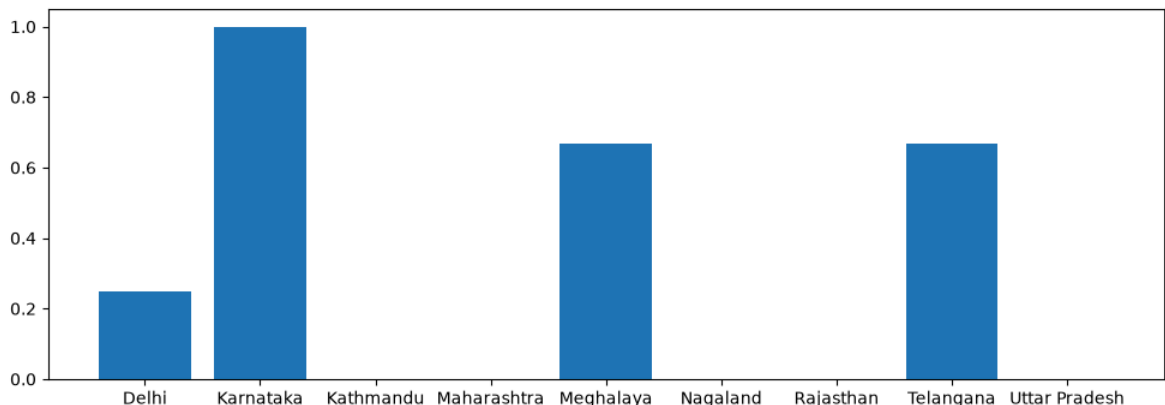
In [54]: *#churn by states*

```
In [55]: churn_plan = df_visual.groupby('state')['Red_flag'].mean()
colors = ['red', 'yellow', 'green']

plt.figure(figsize=(12,4))

plt.bar(churn_plan.index , churn_plan.values)
```

Out[55]: <BarContainer object of 9 artists>



```
In [56]: # wrong method - encoding = conv str to numeric to find correlation

df_encoded = df_visual[['plan_type', 'contract_type', 'Red_flag', 'churn_score', 'churn_risk']]
categorical_cols = ['plan_type', 'contract_type', 'churn_risk']

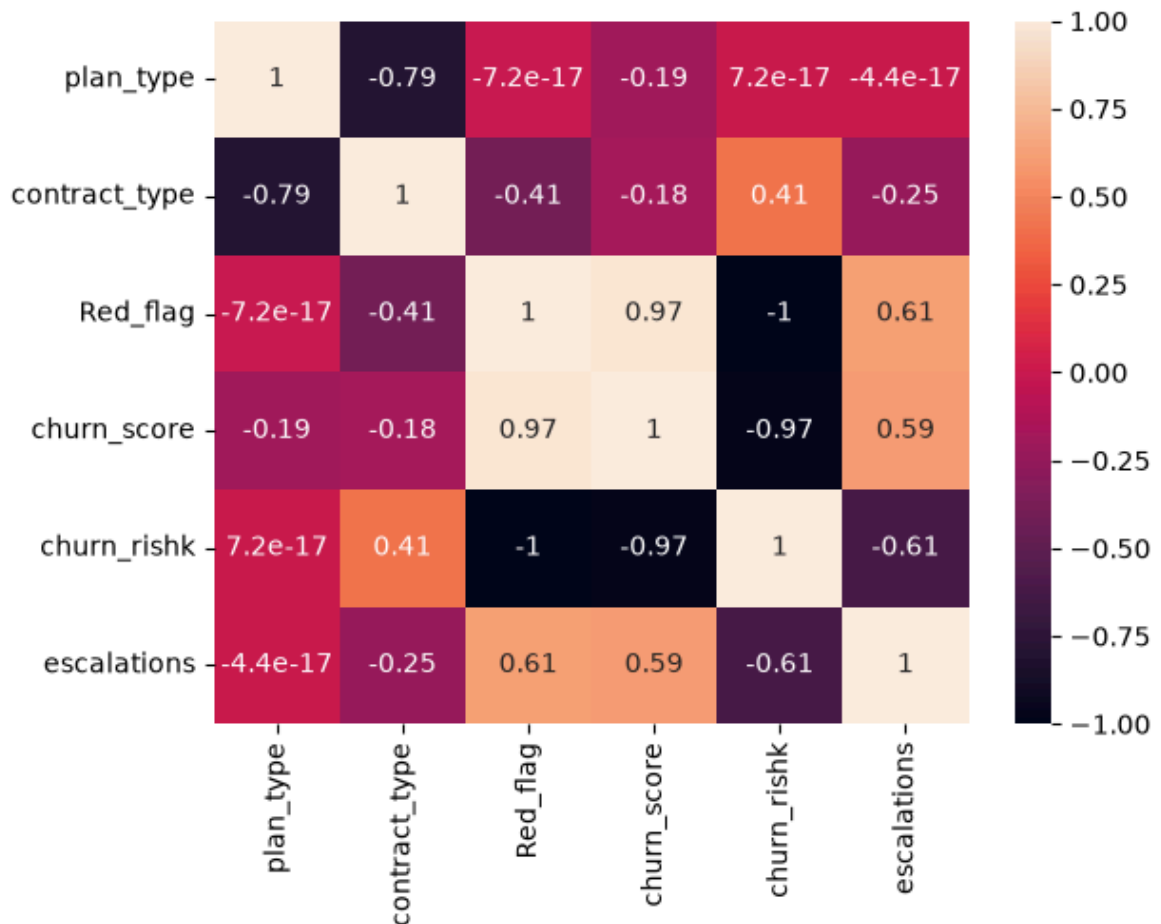
for col in categorical_cols:
    df_encoded[col]=df_encoded[col].astype('category').cat.codes
```

In [57]: *# visualization using seaborn*

In [58]: *# heatmap correlation matrix*

```
sns.heatmap(df_encoded.corr(), annot=True)
```

Out[58]: <Axes: >



```
In [59]: #right method - encoding
df_encoded = df_visual[['plan_type', 'contract_type', 'Red_flag', 'churn_score', 'churn_rishk']]
order_mapping = {'plan_type': ['Basic', 'Standard', 'Premium'],
                 'contract_type': ['Monthly', 'Annual'],
                 'churn_rishk': ['low', 'med', 'high']}

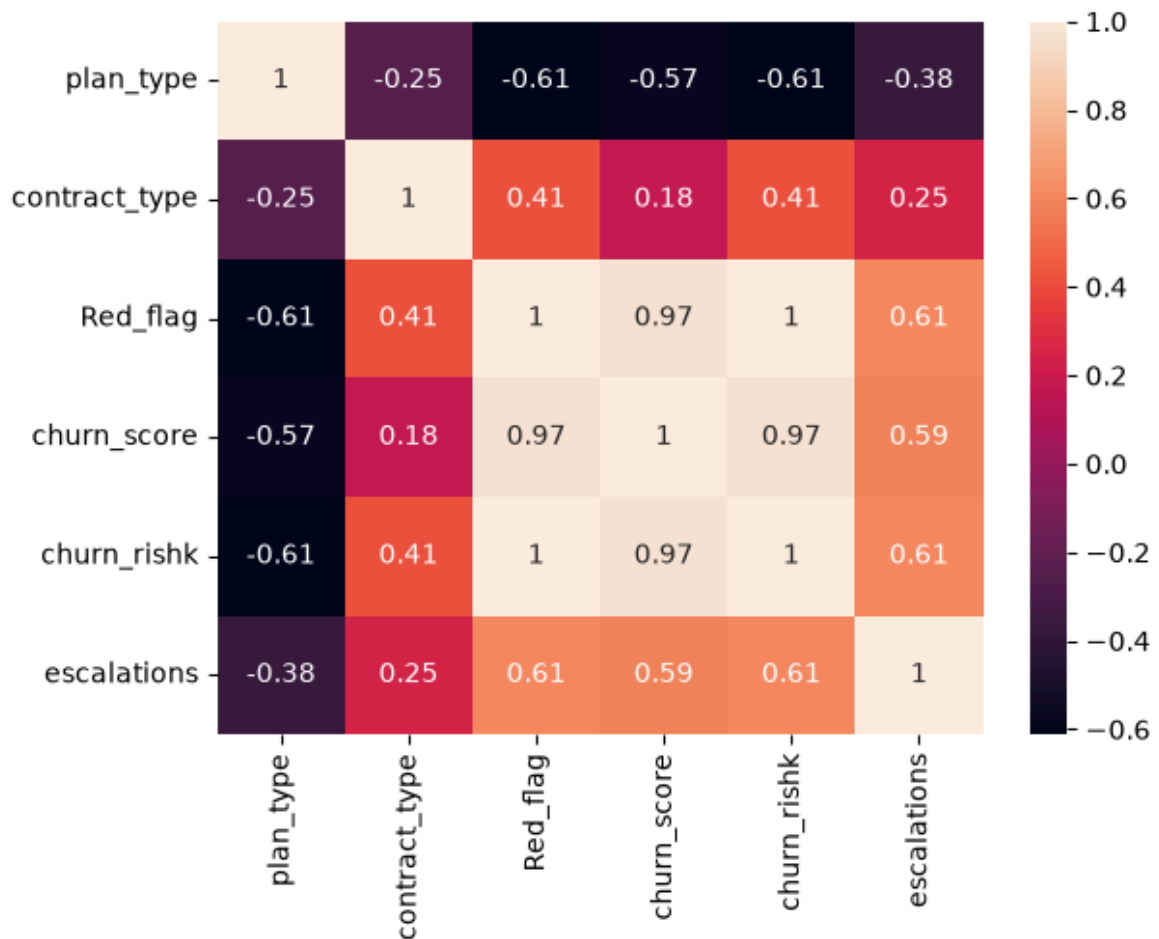
for col, order in order_mapping.items():
    df_encoded[col] = pd.Categorical(df_encoded[col].astype('category'), categories=order, ordered=True)
```

C:\Users\admin\AppData\Local\Temp\ipykernel_11992\2771412949.py:10: Pandas4Warning: Constructing a Categorical with a dtype and values containing non-null entries not in that dtype's categories is deprecated and will raise in a future version.

```
df_encoded[col] = pd.Categorical(df_encoded[col].astype('category'), categories=order, ordered=True).codes
```

```
In [60]: sns.heatmap(df_encoded.corr(), annot=True)
```

Out[60]: <Axes: >

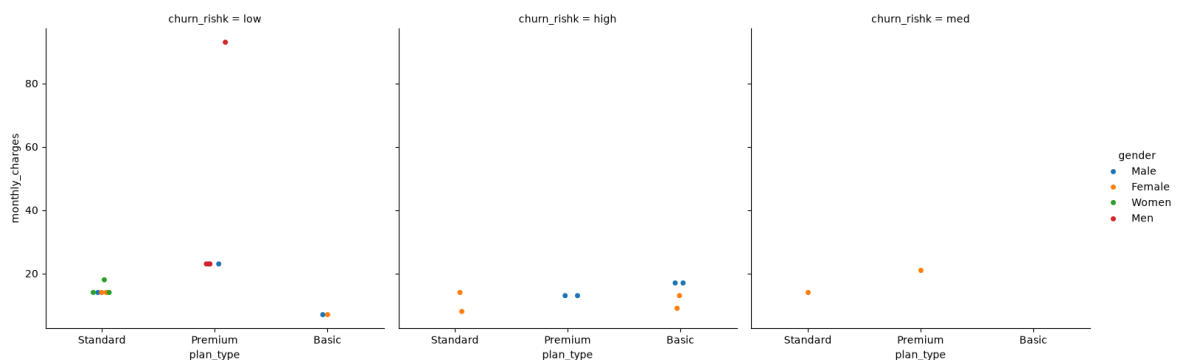


```
In [61]: print(df_visual.columns.tolist())
```

```
['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score', 'Red_flag', 'customer_name', 'country', 'state', 'gender', 'dob', 'complaint_date', 'escalations', 'csat_score', 'complaint_count', 'tenure_days', 'churn_risk', 'cancellation_month']
```

```
In [62]: sns.catplot(data=df_visual,
                    x='plan_type',
                    y='monthly_charges',
                    hue='gender',
                    col = 'churn_risk')
```

```
Out[62]: <seaborn.axisgrid.FacetGrid at 0x20d80712e40>
```



```
In [63]: # pivot table
```

```
In [64]: pd.pivot_table(
    df_visual,
    values='Red_flag',
    index='plan_type',
    aggfunc='mean'
).reset_index()
```

```
Out[64]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>plan_type</th> <th>Red_flag</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> </tbody> <tbody> </tbody>
```

Basic	0.666667
-------	----------

Premium	0.250000
---------	----------

Standard	0.222222
----------	----------

```
In [65]: pd.pivot_table(
    df_visual,
    index='plan_type',
    values=['monthly_charges', 'customerid', 'Red_flag'],
    aggfunc= {
        'monthly_charges': 'sum',
        'customerid': 'nunique',
        'Red_flag': 'mean'
    }
).reset_index()
```

```
Out[65]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>plan_type</th> <th>Red_flag</th> <th>customerid</th>
<th>monthly_charges</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
</tbody> <tbody> </tbody>
```

Basic	0.666667	5	69.94
-------	----------	---	-------

Premium	0.250000	7	231.92
---------	----------	---	--------

Standard	0.222222	9	123.91
----------	----------	---	--------

```
In [66]: # SQL in python
```

```
In [69]: conn = sqlite3.connect('test_database.sqlite')

conn.execute("create table user (first_name text , country text, budget integer)")

conn.commit()
```

```
In [77]: # inserting data

cursor = conn.cursor()
cursor.execute(
    """
    insert into user values
    ('rishi', 'india',5000),
    ('raj', 'germany',3300),
```

```

        ('salar','london',4000)
        """

    )
    conn.commit()
    print("data inserted successfully")

```

data inserted successfully

```

In [78]: conn = sqlite3.connect('test_database.sqlite')
        query = """select * from user """
        df_results = pd.read_sql(query,conn)
        df_results.head(3)

```

```

Out[78]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>first_name</th> <th>country</th> <th>budget</th> </thead>
<tbody> <tr><th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>

```

rishi	india	5000
-------	-------	------

raj	germany	3300
-----	---------	------

salar	london	4000
-------	--------	------

```

In [79]: conn.close()

```

```

In [ ]:

```

```

In [ ]:

```